

Slovak University of Technology in Bratislava

Faculty of Informatics and Information Technologies

NAO ROBOT EXERCISE SYSTEM

Version Control Methodology

Team Leaders

Ing. Jakub Perdek, PhD.
doc. Ing. Fedor Lehocki PhD.

Team Members

Bc. Akhmadinurov Amal
Bc. Danilov Denis
Bc. Dmitriev Anton
Bc. Gainetdinov Ruslan
Bc. Kuzin Timofei
Bc. Tikhoblazhenko Alena
Bc. Zimnitski Aliaksei

Bratislava • 2026/2027

Contents

1 Repository and branch rules 2

2 Changes and commits 2

3 Pull request and integration 3

4 Version numbers and release tags 3

5 Hotfixes, rollback, and exceptions 4

References 5

Project: NAO Robot Exercise System

Scope: Source code, configuration, tests, and project documentation in the NAO repositories.

This document defines how the team records, reviews, integrates, and identifies changes. It expands the version control rules in the *Technical Methodology*. GitHub is the source of truth for code and pull requests (PRs); Jira links the change to a task and its acceptance criteria.

1 Repository and branch rules

1. The `master` branch is the integration branch of `naoMicroservices`. Apply the same workflow to any additional NAO repository, using its agreed integration branch.

2. Do not commit or push directly to the integration branch. A change reaches it through a reviewed PR. Configure GitHub branch protection or a ruleset to require a PR, at least one approval, and the applicable checks where repository permissions allow.

3. Create one short-lived branch per Jira task from the latest integration branch. Name it `<jira-key>-short-description`, for example `NAO-42-video-trimming`. Use the real Jira key; the example is illustrative.

4. Keep branches and PRs focused on one deliverable. If a task needs independent components, split it into linked Jira subtasks and PRs. Avoid long-running shared development branches.

5. A developer may rebase their own branch before review. Do not rewrite a branch another person uses. Never force-push the integration branch or move a published release tag.

2 Changes and commits

1. Pull the current integration branch before starting a task. Commit coherent steps that leave the branch understandable and, where practical, buildable.

2. Write commit subjects as `<type>(<scope>): <summary>`, for example `fix(translator): reject invalid pose data`. Allowed types are `feat`,

`fix`, `test`, `docs`, `refactor`, `build`, and `chore`. Use an imperative, specific summary. Put the Jira key in the PR title or description.

3. Do not commit secrets, local `.env` files, virtual environments, caches, generated video or audio, compiled output, or machine-specific paths. Commit safe example configuration and documented setup steps instead.

4. Update tests, API contracts, configuration examples, and documentation in the same PR as the behavior they describe. Record migrations when a service interface or stored exercise format changes.

5. Inspect `git status` and the staged diff before each push. If a secret is exposed, stop using the credential, rotate it, and coordinate removal from repository history; deleting it in a later commit is insufficient.

3 Pull request and integration

The author links the Jira issue and states the change, reason, test evidence, and limitations. For changes involving motion, safety limits, NAOqi, or camera access, state the hardware impact and the planned robot test. Mark tests as pending when access to NAO is unavailable. Open a draft PR when early feedback is useful; request review only when the change is ready.

At least one teammate other than the author reviews the final diff. A robot behavior or safety change also needs review by a teammate responsible for NAO testing. Resolve blocking comments and rerun relevant checks after changes. The author does not approve their own PR. Follow the review and test gates in the *Technical Methodology*.

Merge with **Squash and merge** after approval, passing checks, and documented verification. The squash commit should retain a descriptive subject and the Jira key. Delete the merged task branch. Update the Jira issue with the PR and result; mark it Done only after the methodology's merge gate is satisfied. If a robot test remains pending, keep that fact visible in Jira and do not enable the change for a live demonstration until the test passes.

4 Version numbers and release tags

Use `MAJOR.MINOR.PATCH` and prefix Git tags with `v`, for example `v0.3.1`. During development use `0.y.z`: increment `MINOR` for a new feature or an incompatible change, and `PATCH` for a compatible fix. The first agreed stable public interface can be `v1.0.0`.

After `1.0.0`, increment `MAJOR` for incompatible interface changes, `MINOR` for compatible functionality, and `PATCH` for compatible fixes. Reset lower components to zero when a higher component changes. This follows Semantic Versioning while giving the team a simple policy for the `0.y.z` phase.

Create a release tag only from a reviewed commit on the integration branch. Use an annotated tag with a short message, for example `git tag -a v0.3.1 -m "NAO exercise demo"`, then push the tag. Do not retarget or reuse a published tag; create a new patch or minor version for corrections. A tag identifies the entire repository state, including service code and Compose configuration. If services live in separate repositories, record each repository's tag or commit SHA in the release notes.

For every demo or physical NAO test, record the tag or full commit SHA, date, operator, robot state, configuration profile, test scenario, and result in the Jira issue or test record. Never record credentials there. If no tag exists yet, use the exact commit SHA. Keep release notes concise: included Jira issues, relevant interface/configuration changes, required migration, and known limitations.

5 Hotfixes, rollback, and exceptions

For an urgent defect, create a task branch from the relevant integration or release commit, link a Jira issue, and use the same review and verification gate. When a live demo must return to a known-good state, redeploy the last verified tag or commit and record the action and reason. Do not solve a failed release by rewriting Git history or moving its tag.

If GitHub checks are unavailable, record the equivalent local commands and results in the PR. Lack of physical NAO access is not a reason to skip offline tests; schedule the hardware test and keep it pending visibly. Any exception to this workflow must be documented in the PR and agreed by the team before merge.

References

[1] S. Chacon and B. Straub. "Pro git: Branching workflows." (), [Online]. Available: https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows (visited on 10/22/2026).

[2] S. Chacon and B. Straub. "Pro git: Tagging." (), [Online]. Available: https://git-scm.com/book/en/v2/Git-Basics-Tagging (visited on 10/22/2026).

[3] GitHub. "Pull request merges." (), [Online]. Available: https://docs.github.com/en/pull-requests/reference/pull-request-merges (visited on 10/22/2026).

[4] GitHub. "About rulesets." (), [Online]. Available: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets (visited on 10/22/2026).

[5] T. Preston-Werner. "Semantic versioning 2.0.0." (), [Online]. Available: https://semver.org/ (visited on 10/22/2026).